

CS 141

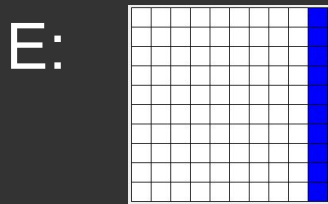
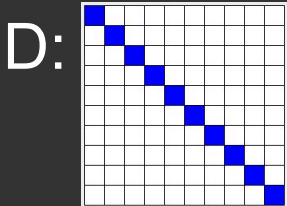
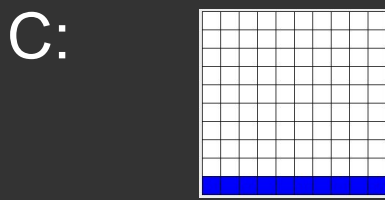
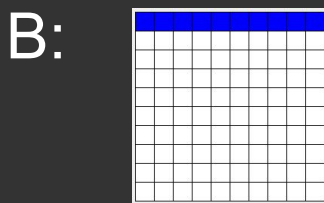
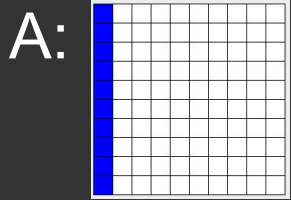
Flagmaker

Clicker Questions

and examples

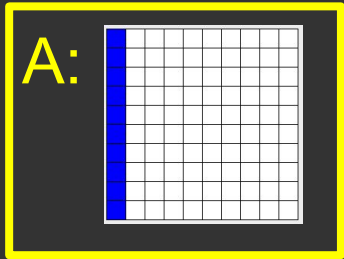
What does this loop draw?

```
for (int i=0; i<grid.getNumRows(); i++) {  
    grid.setColor(i, 0, Color.BLUE);  
}
```

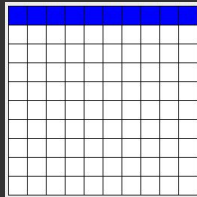


What does this loop draw?

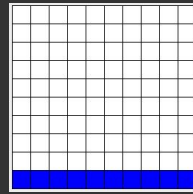
```
for (int i=0; i<grid.getNumRows(); i++) {  
    grid.setColor(i, 0, Color.BLUE);  
}
```



B:

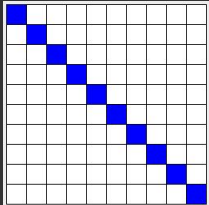


C:

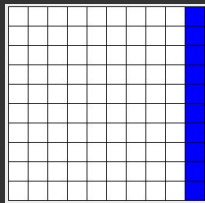


Paint the first column of
all rows blue

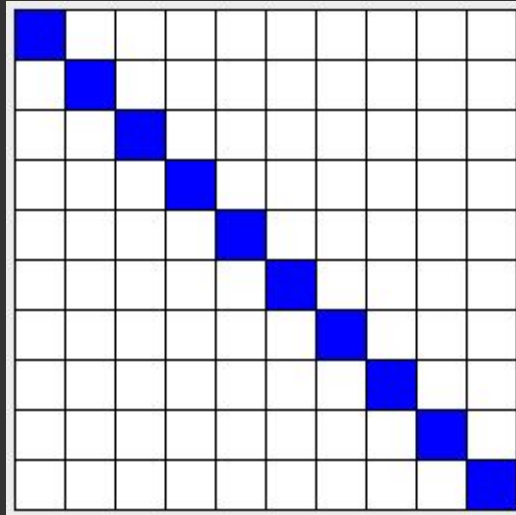
D:



E:



Let's draw a diagonal line!



Draw a diagonal line

A

```
for (int i=0; i<g.getNumRows(); i++) {  
    g.setColor(i, i, Color.BLUE);  
}
```

B

```
for (int i=0; i<g.getNumRows(); i++) {  
    g.setColor(0, i, Color.BLUE);  
}
```

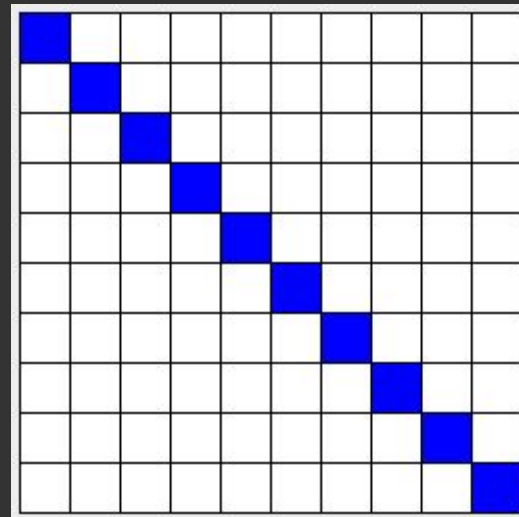
C

```
for (int i=g.getNumRows()-1; i>=0; i--) {  
    g.setColor(i, i, Color.BLUE);  
}
```

D

```
for (int i=g.getNumRows(); i>0; i--) {  
    g.setColor(i, i, Color.BLUE);  
}
```

E Not exactly one of the above



Draw a diagonal line

A

```
for (int i=0; i<g.getNumRows(); i++) {  
    g.setColor(i, i, Color.BLUE);  
}
```

B

```
for (int i=0; i<g.getNumRows(); i++) {  
    g.setColor(0, i, Color.BLUE);  
}
```

C

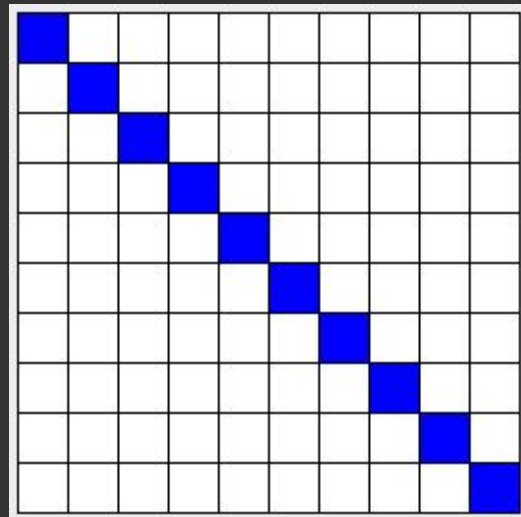
```
for (int i=g.getNumRows()-1; i>=0; i--) {  
    g.setColor(i, i, Color.BLUE);  
}
```

D

```
for (int i=g.getNumRows(); i>0; i--) {  
    g.setColor(i, i, Color.BLUE);  
}
```

E Not exactly one of the above

Two correct answers!
Plot twist!



Which is better?

What colors the whole grid blue?

A

```
for (int j=0; j<grid.getNumCols(); j++) {  
    for (int i=0; i<grid.getNumRows(); i++) {  
        grid.setColor(i, j, Color.BLUE);  
    }  
}
```

B

```
for (int j=0; j<grid.getNumCols(); j++) {  
    int i=j;  
    grid.setColor(j, i, Color.BLUE);  
}
```

C

```
for (int c=0; c<grid.getNumCols(); c++) {  
    for (int r=0; r<grid.getNumRows(); r++) {  
        grid.setColor(c, r, Color.BLUE);  
    }  
}
```

D all of them

E none of them

What colors the whole grid blue?

A

```
for (int j=0; j<grid.getNumCols(); j++) {  
    for (int i=0; i<grid.getNumRows(); i++) {  
        grid.setColor(i, j, Color.BLUE);  
    }  
} // fills in grid, one column at a time
```

B

```
for (int j=0; j<grid.getNumCols(); j++) {  
    int i=j;  
    grid.setColor(j, i, Color.BLUE);  
} // fills in the diagonal only (all i=j spots)
```

C

```
for (int c=0; c<grid.getNumCols(); c++) {  
    for (int r=0; r<grid.getNumRows(); r++) {  
        grid.setColor(c, r, Color.BLUE);  
        //setColor expects row, col, Color  
    }  
} // will most likely crash with indexOutOfBounds
```

D all of them

E none of them

**Note that row & column are switched around!
setColor() interprets first argument as row and second as column, regardless of what we meant!**

Either will color the entire grid blue! What's different?

```
for (int j=0; j<grid.getNumCols(); j++) {  
    for (int i=0; i<grid.getNumRows(); i++) {  
        grid.setColor(i, j, Color.BLUE);  
    }  
}
```

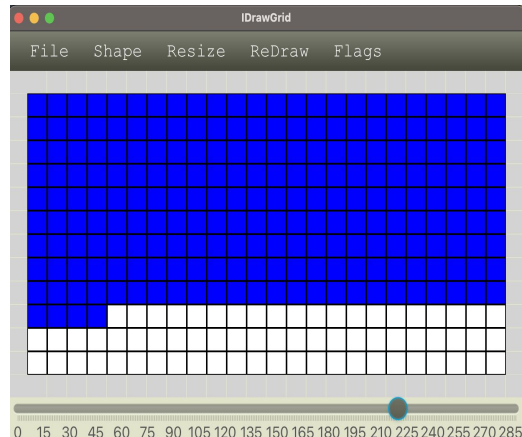
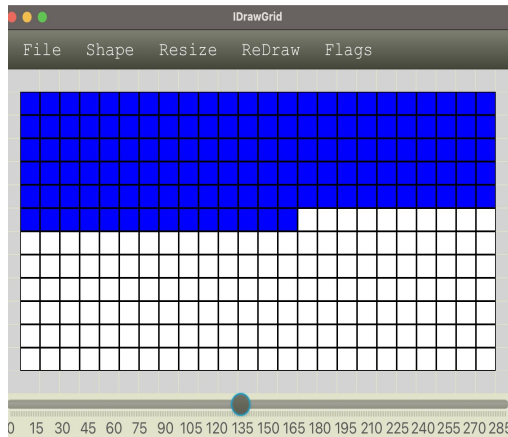
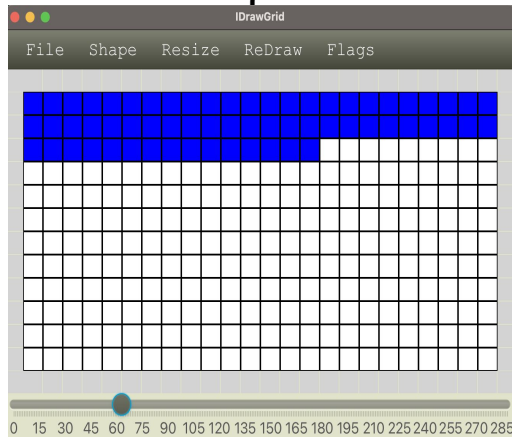
```
for (int i=0; i<grid.getNumRows(); i++) {  
    for (int j=0; j<grid.getNumCols(); j++) {  
        grid.setColor(i, j, Color.BLUE);  
    }  
}
```

AllBlues1 — try the replay thumb at bottom

```
@Flag(name = "AllBlues1", num = 5)
public static void allBlues1(IDrawGrid grid) {
    for (int r=0; r<grid.getNumRows(); r++) {
        for (int c=0; c<grid.getNumCols(); c++) {
            grid.setColor(r, c, Color.BLUE);
        }
    }
}
```

AllBlues1 — try the replay thumb at bottom

```
@Flag(name = "AllBlues1", num = 5)
public static void allBlues1(IDrawGrid grid) {
    for (int r=0; r<grid.getNumRows(); r++) {
        for (int c=0; c<grid.getNumCols(); c++) {
            grid.setColor(r, c, Color.BLUE);
        }
    }
}
```

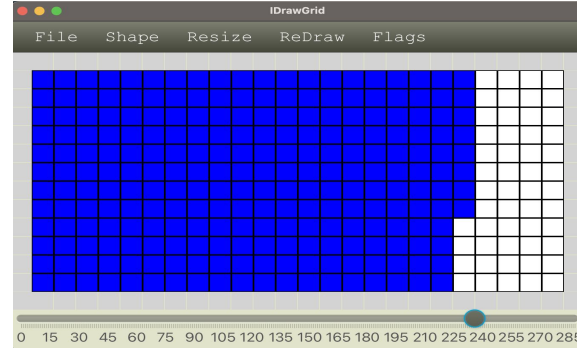
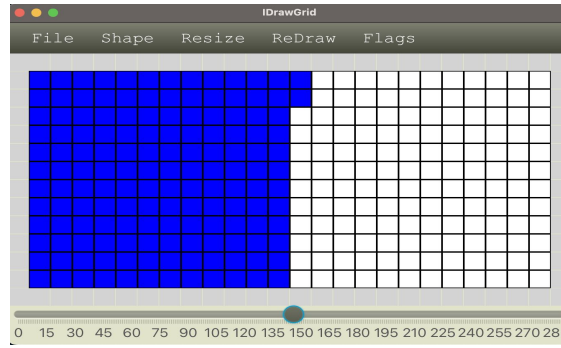
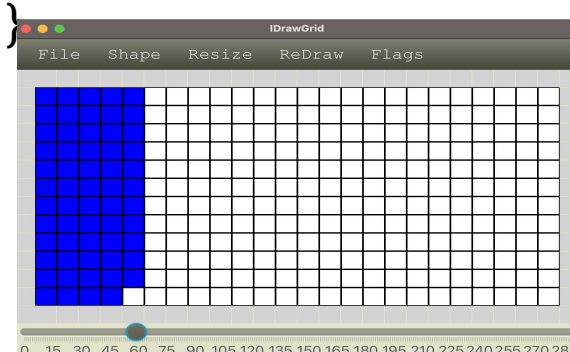


AllBlues2 — again replay at the bottom

```
@Flag(name = "AllBlues2", num = 6)
public static void allBlues2(IDrawGrid grid) {
    for (int c=0; c<grid.getNumCols(); c++) {
        for (int r=0; r<grid.getNumRows(); r++) {
            grid.setColor(r, c, Color.BLUE);
        }
    }
}
```

AllBlues2 — again replay at the bottom

```
@Flag(name = "AllBlues2", num = 6)
public static void allBlues2(IDrawGrid grid) {
    for (int c=0; c<grid.getNumCols(); c++) {
        for (int r=0; r<grid.getNumRows(); r++) {
            grid.setColor(r, c, Color.BLUE);
        }
    }
}
```



Either will color the entire grid blue! What's different?

```
for (int j=0; j<grid.getNumCols(); j++) {  
    for (int i=0; i<grid.getNumRows(); i++) {  
        grid.setColor(i, j, Color.BLUE);  
    }  
}
```

for each j, we will color all i cells from 0 to last row

- for each column, we'll color all the rows
- thus, we will color one column at a time

```
for (int i=0; i<grid.getNumRows(); i++) {  
    for (int j=0; j<grid.getNumCols(); j++) {  
        grid.setColor(i, j, Color.BLUE);  
    }  
}
```

for each i, we will color all j cells from 0 to last column

- for each row, we will color all the columns
- thus, we will color one row at a time

What squares will the code below color?

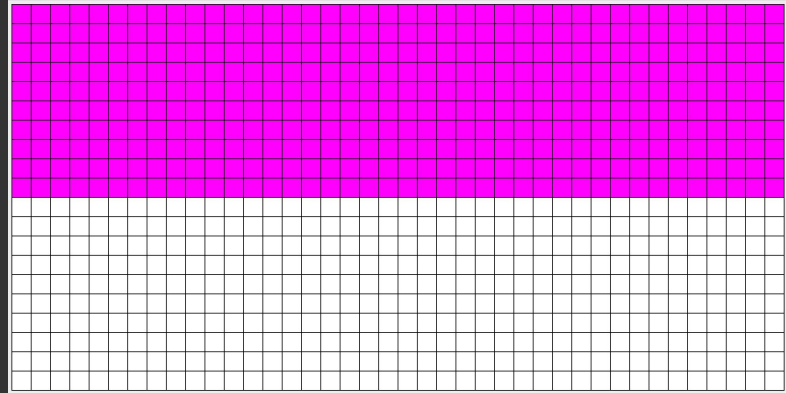
```
for (int i = 0; i < grid.getNumRows()/2; i++) {  
    for (int j = 0; j < grid.getNumCols(); j++) {  
        grid.setColor(i, j, Color.MAGENTA);  
    }  
}
```

- A) The left half of the grid
- B) The right half of the grid
- C) The top half of the grid
- D) The bottom half of the grid
- E) all of the above

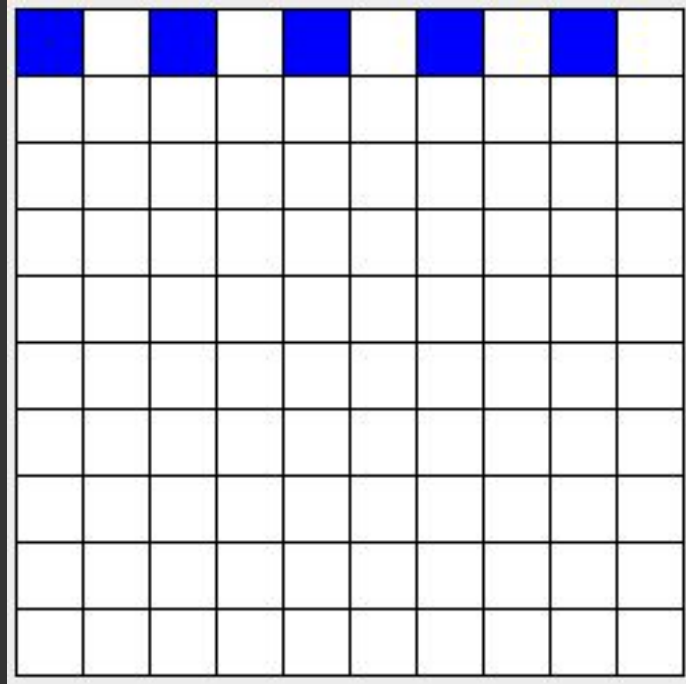
What squares will the code below color?

```
for (int i = 0; i < grid.getNumRows()/2; i++) {  
  //top half of rows  
    for (int j = 0; j < grid.getNumCols(); j++) {  
      //all the columns  
        grid.setColor(i, j, Color.MAGENTA);  
    }  
}
```

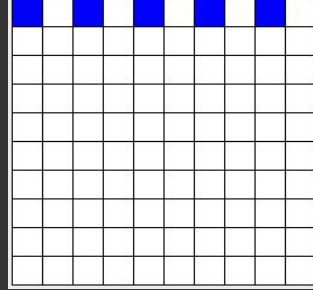
- A. The left half of the grid
- B. The right half of the grid
- C. The top half of the grid
- D. The bottom half of the grid
- E. all of the above



How do we draw a dashed blue line across the top of the grid?



Which will draw a dashed blue line across the top of the grid?



```
for (int i=0; i<g.getNumCols(); i++) {  
    g.setColor(0, i, Color.BLUE);  
}
```

A

```
for (int i=1; i<g.getNumCols(); i+=2) {  
    g.setColor(0, i, Color.BLUE);  
}
```

B

```
for (int i=0; i<g.getNumCols(); i++) {  
    if (i % 2 == 0) g.setColor(0, i, Color.BLUE);  
}
```

C

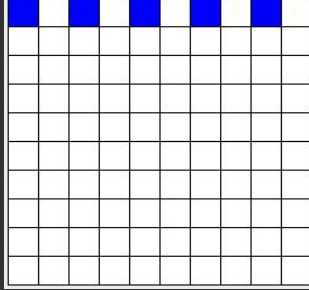
```
for (int i=0; i<g.getNumCols(); i+=2) {  
    if (i % 2 == 1) g.setColor(0, i, Color.BLUE);  
}
```

D

Not exactly one of the above

E

Which will draw a dashed blue line across the top of the grid?



```
for (int i=0; i<g.getNumCols(); i++) {  
    g.setColor(0, i, Color.BLUE);  
}
```

A

```
for (int i=1; i<g.getNumCols(); i+=2) {  
    g.setColor(0, i, Color.BLUE);  
}
```

B

```
for (int i=0; i<g.getNumCols(); i++) {  
    if (i % 2 == 0) g.setColor(0, i, Color.BLUE);  
}
```

C

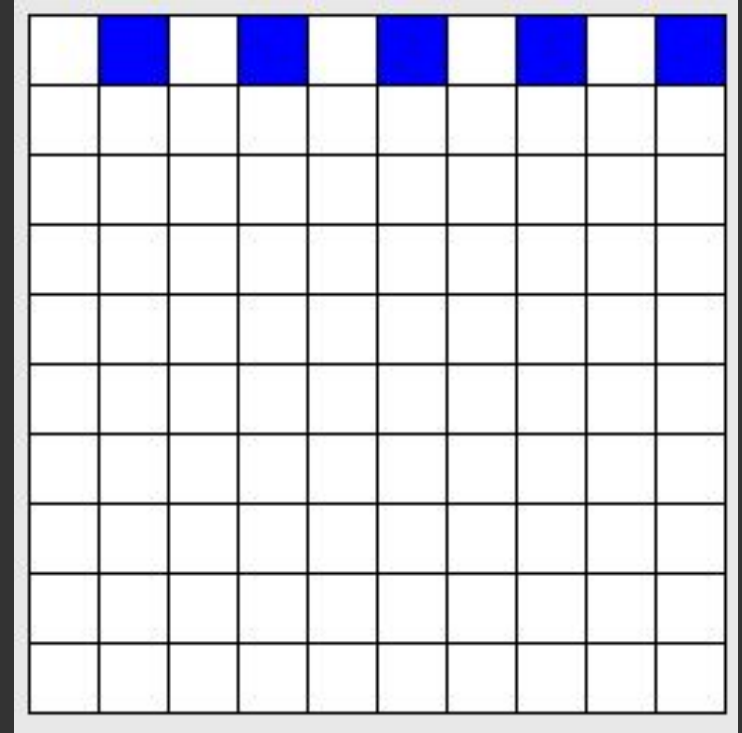
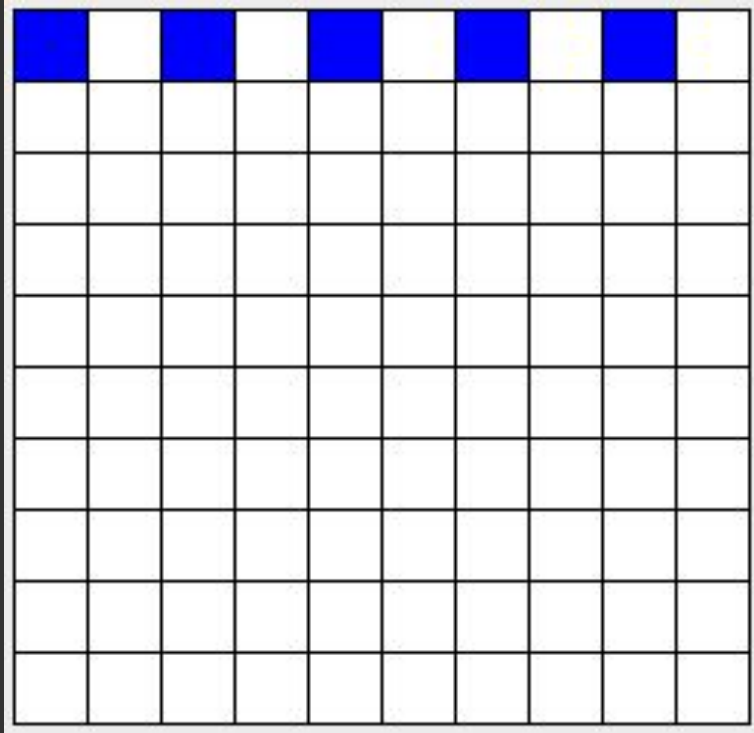
```
for (int i=0; i<g.getNumCols(); i+=2) {  
    if (i % 2 == 1) g.setColor(0, i, Color.BLUE);  
}
```

D

Not exactly one of the above

E

How do we start the dashed line at the second square instead?



A 10x10 grid with the top row containing alternating blue and white squares, starting and ending with a blue square.

A

B

C

D

E

How do we start the dashed line in top row at the second square?

```
for (int i=1; i<g.getNumCols(); i++) {  
    g.setColor(0, i, Color.BLUE);  
}
```

A

```
for (int i=0; i<g.getNumCols(); i+=2) {  
    g.setColor(0, i, Color.BLUE);  
}
```

B

```
for (int i=0; i<g.getNumCols(); i++) {  
    if (i % 2 == 1) g.setColor(0, i, Color.BLUE);  
}
```

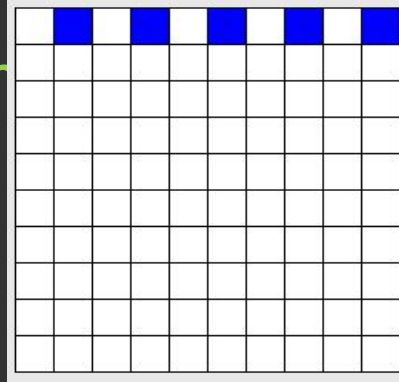
C

```
for (int i=1; i<g.getNumCols(); i+=2) {  
    g.setColor(0, i, Color.BLUE);  
}
```

D

Not exactly one of the above

E



What squares will the code below color?

```
// assume grid is 20x20
for (int i=grid.getNumRows()-1; i>=0; i--) {
    grid.setColor(i, i, Color.GREEN);
}
```

- A. the main diagonal (upper left to lower right)
- B. the backward diagonal (upper right to lower left)
- C. the middle column, top to bottom
- D. the middle row, left to right
- E. not exactly one of the above

What squares will the code below color?

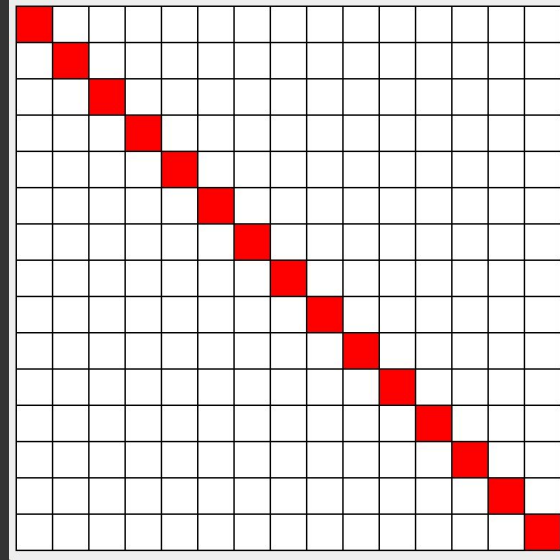
```
// assume grid is 20x20
for (int i=grid.getNumRows()-1; i>=0; i--) {
    grid.setColor(i, i, Color.GREEN);
}
```

- A. the main diagonal (upper left to lower right)
- B. the backward diagonal (upper right to lower left)
- C. the middle column, top to bottom
- D. the middle row, left to right
- E. not exactly one of the above

What draws a diagonal?

```
// assume grid is 15x15  
for (int i=0; i<grid.getNumRows(); i++) {  
    _____  
}
```

- A. `grid.setColor(i, i, Color.RED);`
- B. `grid.setColor(i, 0, Color.RED);`
- C. `grid.setColor(0, i, Color.RED);`
- D. `grid.setColor(grid.getNumRows()-i-1, grid.getNumRows()-i-1, Color.RED);`
- E. not exactly one of the above

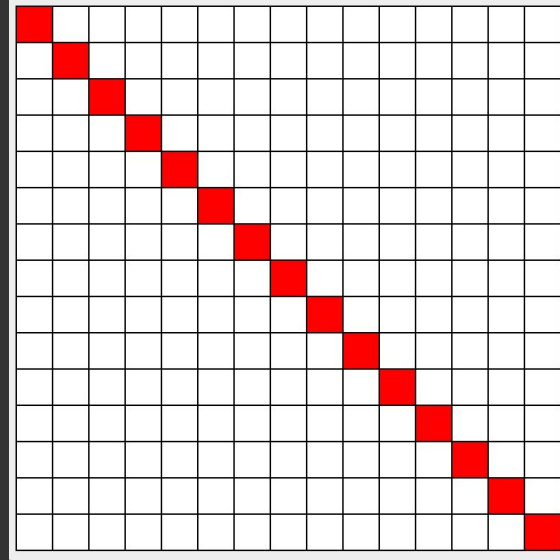


What draws a diagonal?

```
// assume grid is 15x15  
for (int i=0; i<grid.getNumRows(); i++) {  
    _____  
}
```

- A. grid.setColor(i, i, Color.RED);
- B. grid.setColor(i, 0, Color.RED);
- C. grid.setColor(0, i, Color.RED);
- D. grid.setColor(grid.getNumRows()-i-1, grid.getNumRows()-i-1, Color.RED);
- E. not exactly one of the above

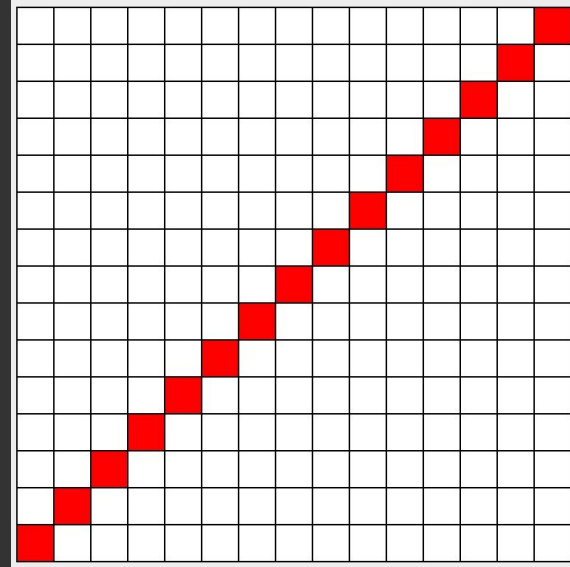
Two (well, three)
right answers!



Let's draw a reverse diagonal!

```
// assume grid is 15x15  
for (int i=0; i<grid.getNumRows(); i++) {  
    _____  
}
```

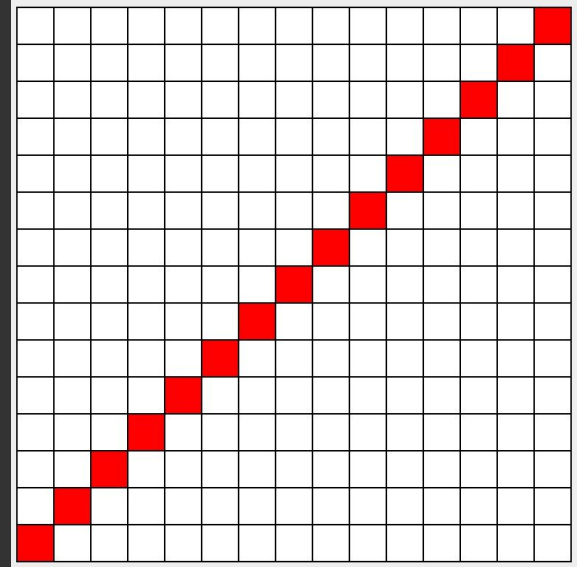
- A. `grid.setColor(i, i, Color.RED);`
- B. `grid.setColor(i, 0, Color.RED);`
- C. `grid.setColor(0, i, Color.RED);`
- D. `grid.setColor(i, grid.getNumRows()-i-1, Color.RED);`
- E. not exactly one of the above



Let's draw a reverse diagonal!

```
// assume grid is 15x15  
for (int i=0; i<grid.getNumRows(); i++) {  
    _____  
}
```

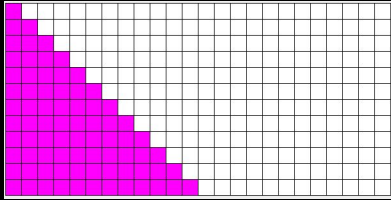
- A. `grid.setColor(i, i, Color.RED);`
- B. `grid.setColor(i, 0, Color.RED);`
- C. `grid.setColor(0, i, Color.RED);`
- D. `grid.setColor(i, grid.getNumRows()-i-1, Color.RED);`**
- E. not exactly one of the above



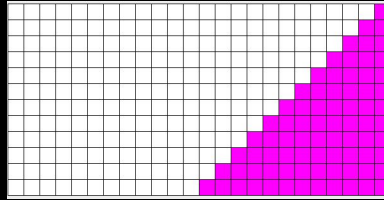
What squares will the code below color?

```
// assume grid is 10x20  
for (int i = 0; i < grid.getNumRows(); i++) {  
    for (int j = 0; j <= (2*i+1); j++) {  
        grid.setColor(i, j, Color.MAGENTA);  
    }  
}
```

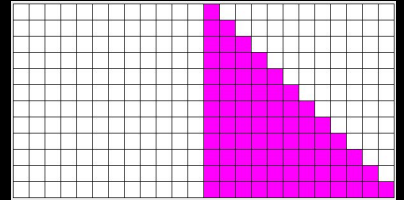
A



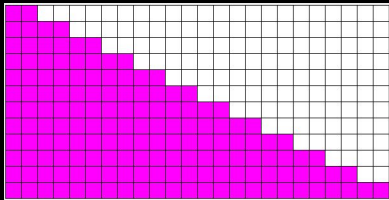
C



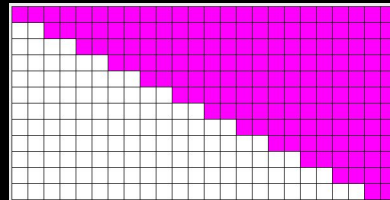
E



B



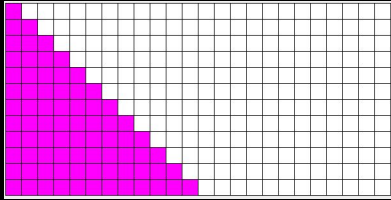
D



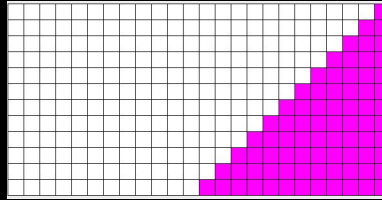
What squares will the code below color?

```
// assume grid is 10x20  
for (int i = 0; i < grid.getNumRows(); i++) { //all the rows  
    for (int j = 0; j <= (2*i+1); j++) { // from col [0] to col [2*row+1],  
inclusively  
        grid.setColor(i, j, Color.MAGENTA);  
    }  
}
```

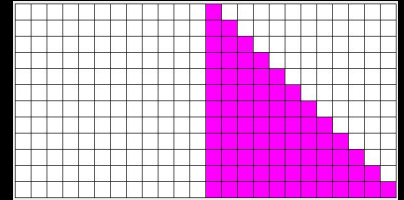
A



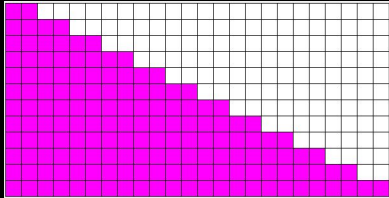
C



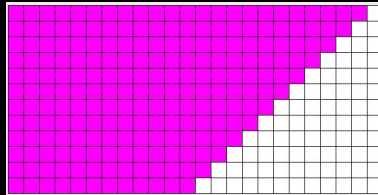
E



B



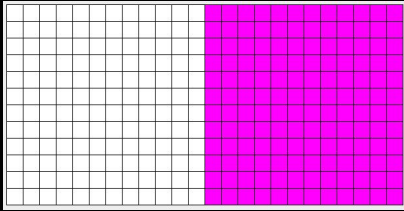
D



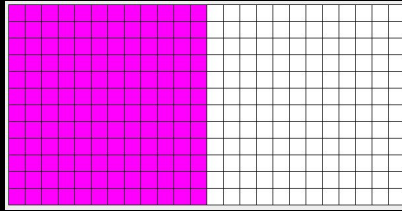
What squares will the code below color?

```
// assume grid is 12x24  
for (int i=g.getNumRows()/2; i<g.getNumRows(); i++) {  
    for (int j=g.getNumCols()/2; j<g.getNumCols(); j++) {  
        g.setColor(i, j, Color.MAGENTA);  
    }  
}
```

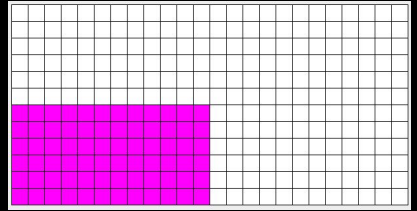
A



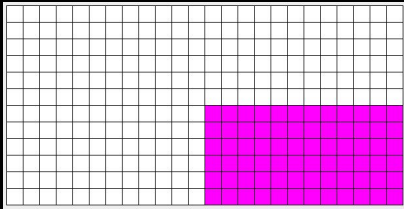
C



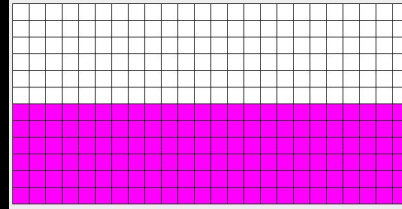
E



B



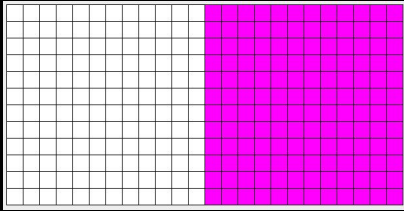
D



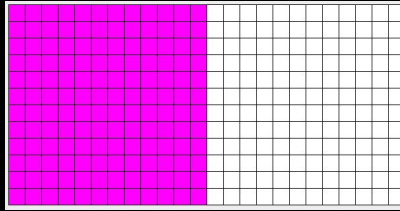
What squares will the code below color?

```
// assume grid is 12x24  
for (int i=g.getNumRows()/2; i<g.getNumRows(); i++) { //from half rows to end  
    for (int j=g.getNumCols()/2; j<g.getNumCols(); j++) { //from half cols to end  
        g.setColor(i, j, Color.MAGENTA);  
    }  
}
```

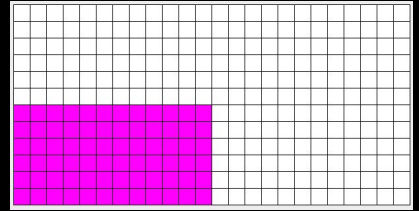
A



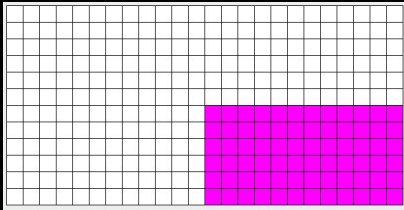
C



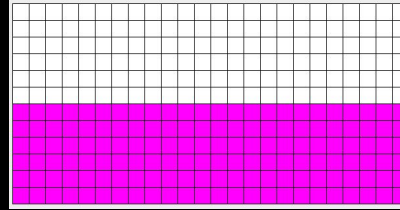
E



B

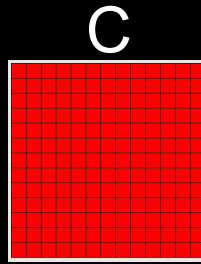
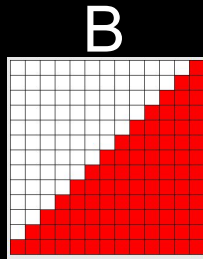
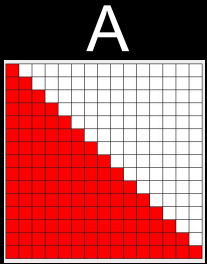


D



What does this draw (assume square grid)?

```
for (int r=0; r<grid.getNumRows(); r++){  
    for (int c=0; c<=r; c++) {  
        grid.setColor(r, c, Color.RED);  
    }  
}
```

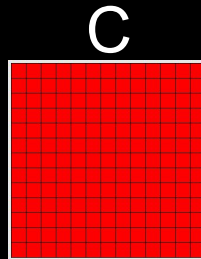
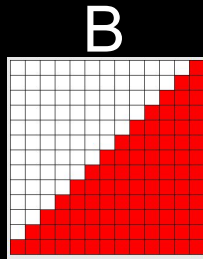
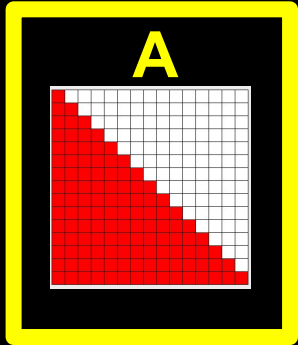


D: it crashes

E: none of these

What does this draw (assume square grid)?

```
for (int r=0; r<grid.getNumRows(); r++){ // from row[0] to end (all rows)
    for (int c=0; c<=r; c++) { // from col[0] to col[row]
        grid.setColor(r, c, Color.RED);
    }
}
```

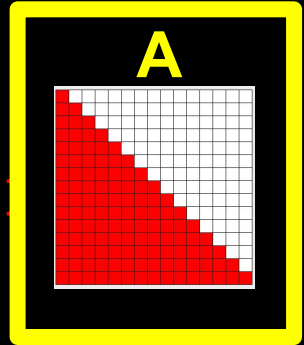


D: it crashes

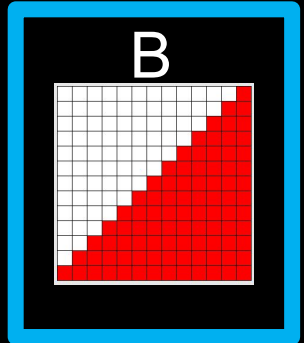
E: none of these

Mirroring a position

```
for (int r=0; r<grid.getNumRows(); r++){ // from row[0] to end
    for (int c=0; c<=r; c++) { // from col[0] to col[row]
        grid.setColor(r, c, Color.RED); // this row[r] and col[c]
    }
}
```



```
for (int r=0; r<grid.getNumRows(); r++){
    for (int c=0; c<=r; c++) {
        grid.setColor(r, grid.getNumCols()-1-c, Color.RED);
        //this row[r], mirror col: [lastCol-currentCol]
        //this row[r], mirror col: [(allColumns-1) - c]
    }
}
```



Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]

GIVEN SOME INDEX [i]



[7] [8] [9] [10] [11] [12] [13]

WHAT'S THE MIRROR INDEX ??

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

first = [0]

second = [1]

third = [2]

[i]

MIRROR OF INDEX [i]

[???]

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

first = [0]

second = [1]

third = [2]

[i]

MIRROR OF INDEX [i]

last = [13] = [totalCount-1]

[???]

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

first = [0]

second = [1]

third = [2]

[i]

MIRROR OF INDEX [i]

last = [13] = [totalCount-1]

second to last = [12] = [totalCount-2]

[???]

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

first = [0]

second = [1]

third = [2]

[i]

MIRROR OF INDEX [i]

last = [13] = [totalCount-1]

second to last = [12] = [totalCount-2]

third to last = [11] = [totalCount-3]

[???]

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

first = [0] = first + 0

second = [1] = first + 1

third = [2] = first + 2

[i]

MIRROR OF INDEX [i]

last = [13] = [totalCount-1] = last - 0

second to last = [12] = [totalCount-2] = last - 1

third to last = [11] = [totalCount-3] = last - 2

[???]

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

First = [0] = first + 0

Second = [1] = first + 1

Third = [2] = first + 2

Some [i] = first + i

MIRROR OF INDEX [i]

Last = [13] = [totalCount-1] = last - 0

Second to last = [12] = [totalCount-2] = last - 1

Third to last = [11] = [totalCount-3] = last - 2

Mirror of [i] = last - i

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

First = [0] = first + 0

Second = [1] = first + 1

Third = [2] = first + 2

MIRROR OF INDEX [i]

Last = [13] = [totalCount-1] = last - 0

Second to last = [12] = [totalCount-2] = last - 1

Third to last = [11] = [totalCount-3] = last - 2

Some [i] = first + i = 0 + i = [i]

Mirror of [i] = last - i = [totalCount - 1 - i]

Let's mirror the K-pop band BTS



[0] [1] [2] [3] [4] [5] [6]



[7] [8] [9] [10] [11] [12] [13]

SOME INDEX [i]

Some [i] = **first** + i = 0 + i = [i]

MIRROR OF INDEX [i]

Mirror of [i] = **last** - i = [totalCount - 1 - i]

Example: given [4], mirror is at [totalCount-1-4] = [14-1-4] = [9]

given some **current** index, its mirror is always at
 $\text{totalSpots}-1-\text{current}$

start =
0

current
spot

mirror
spot

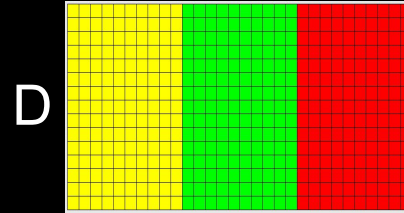
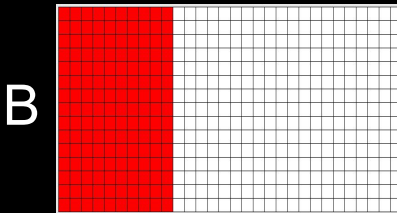
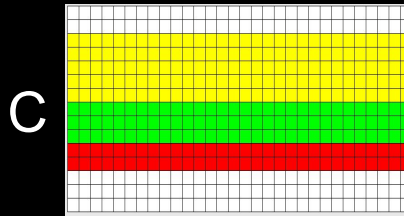
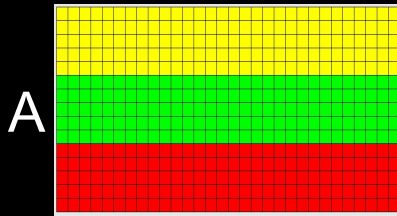
end =
 $\text{totalSpots}-1$

current spots forward from start =
 $0 + \text{current} = \text{current}$

current spots backward from end =
 $\text{totalSpots}-1-\text{current}$

What does this create (assume rectangular grid)

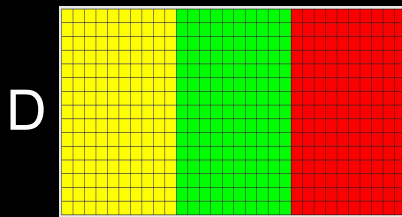
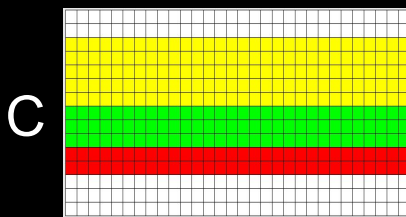
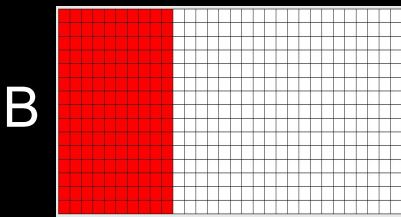
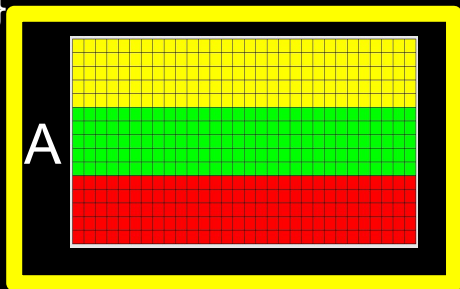
```
for (int r=0; r<grid.getNumRows()/3; r++) {  
    for (int c=0; c<grid.getNumCols(); c++) {  
        grid.setColor(r, c, Color.YELLOW);  
        grid.setColor(r + grid.getNumRows()/3, c, Color.GREEN);  
        grid.setColor(r + 2*grid.getNumRows()/3, c, Color.RED);  
    }  
}
```



E: none of the above

What does this create (assume rectangular grid)

```
for (int r=0; r<grid.getNumRows()/3; r++) {  
    for (int c=0; c<grid.getNumCols(); c++) {  
        grid.setColor(r, c, Color.YELLOW);  
        grid.setColor(r + grid.getNumRows()/3, c, Color.GREEN);  
        grid.setColor(r + 2*grid.getNumRows()/3, c, Color.RED);  
    }  
}
```

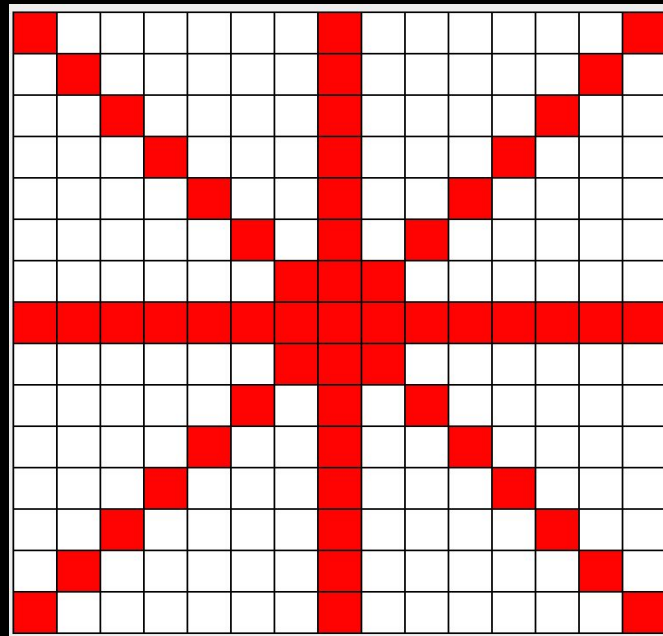


“Lithuania with only one loop!
Is that legal? Shouldn't
programming power like that be a
sacred and controlled knowledge?
Shouldn't it cost extra for that?
I can't believe you're giving this
information away!” - you, probably

E: none of these

What is the *minimum* number of loops required to draw this pattern on a square grid?

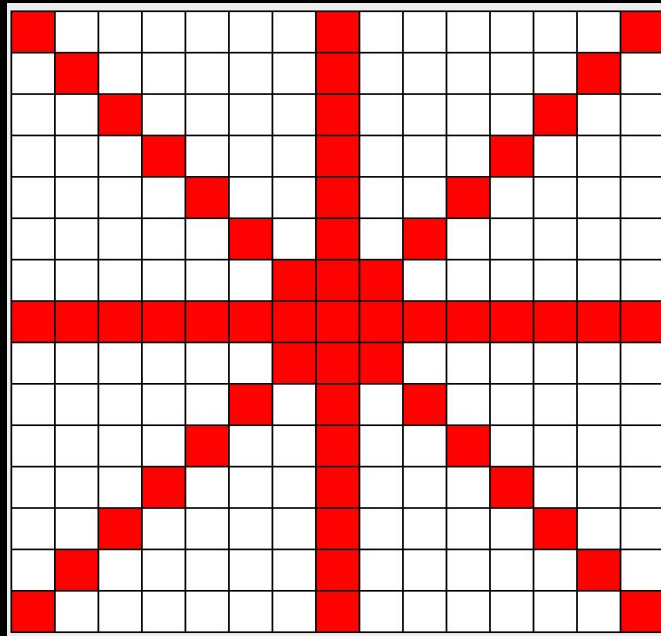
- A. 0
- B. 1
- C. 2
- D. 3
- E. 4



What is the *minimum* number of loops required to draw this pattern on a square grid?

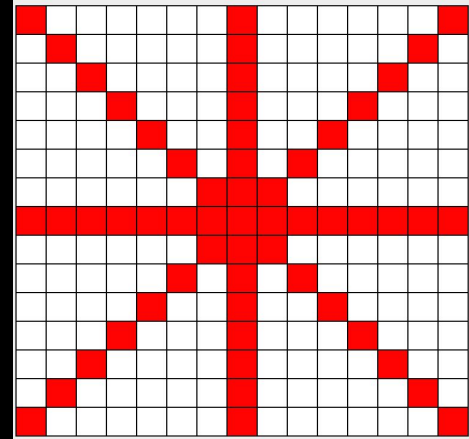
- A. 0
- B. 1
- C. 2
- D. 3
- E. 4

You might envision one loop per line drawn.
We could do that, but how many spots do we paint in each line in an $(n \times n)$ grid? N spots...
So we could just go through a single loop of n iterations and paint a spot in each of our lines...



What is the *minimum* number of loops required to draw this pattern on a square grid?

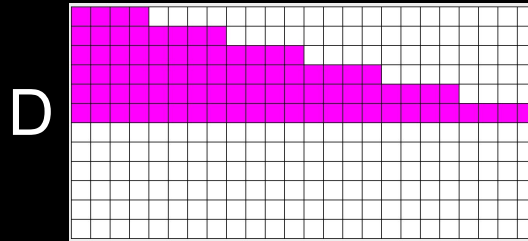
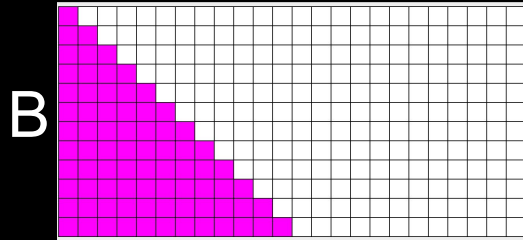
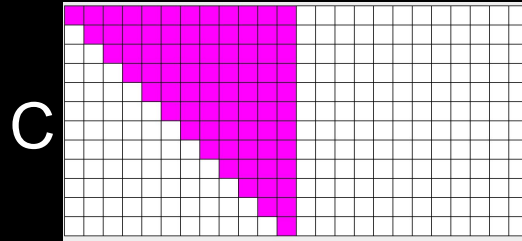
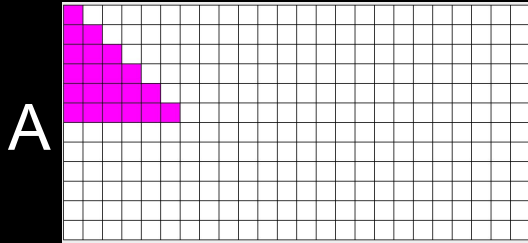
- A. 0
- B. 1
- C. 2
- D. 3
- E. 4



```
int size=15; //could be any value >=5
SimpleGrid grid = new SimpleGrid(size);
for (int i=0; i<size; i++) {
    grid.setColor(i, i, Color.RED); //diagonal top-left to bottom-right
    grid.setColor(i, size-i-1, Color.RED); // diagonal top-right to bottom-left
    grid.setColor(i, size/2, Color.RED); // every row for middle column
    grid.setColor(size/2, i, Color.RED); // middle row for every column
}
```

What does this create (assume rectangular grid)

```
int n=4;
for (int i=0; i<g.getNumRows()/2; i++) {
    for (int j=0; j<(i+1)*n; j++) {
        g.setColor(i, j, Color.magenta);
    }
}
```



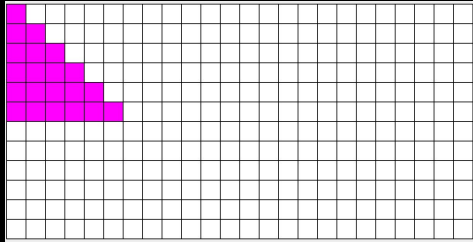
E: none of these

What does this create (assume rectangular grid)

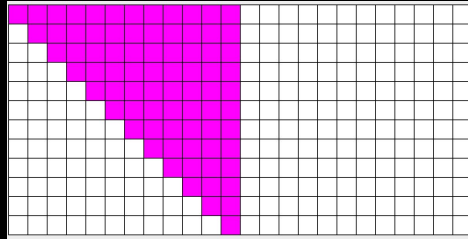
```
int n=4;
for (int i=0; i<g.getNumRows()/2; i++) { //from row [0] to half
    for (int j=0; j<(i+1)*n; j++) { // from col [0] up to col [(row+1)*4]
        g.setColor(i, j, Color.magenta);
    }
}
```

So for row 0 we'll have 4 cols
For row 1 we'll have 8 cols,
etc.

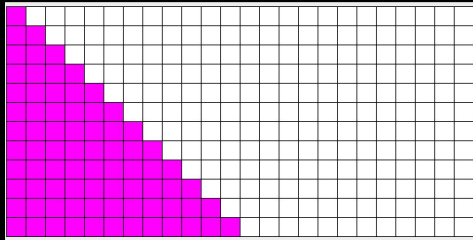
A



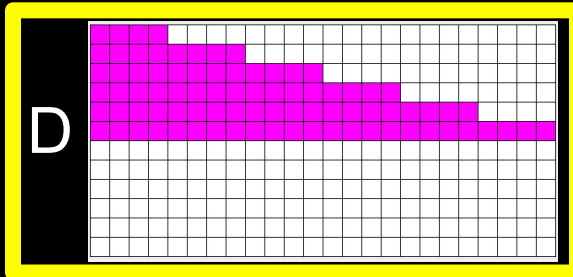
C



B



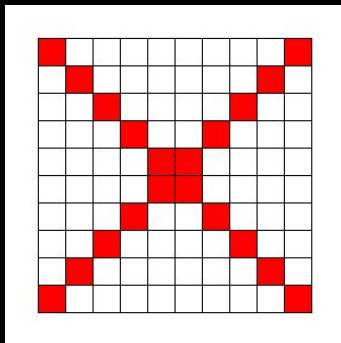
D



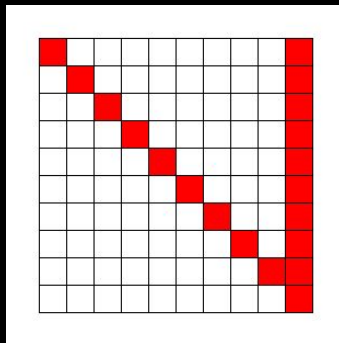
E: none of these

What will this draw?

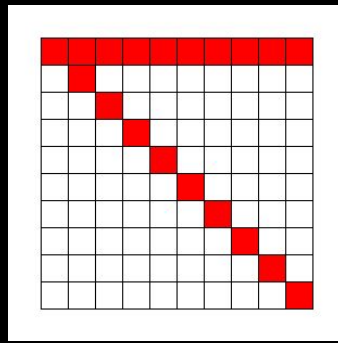
```
int size = 10;  
SimpleGrid g=new SimpleGrid(size, size);  
for (int i=0; i < size; i++)  
    g.setColor(i, i, Color.RED);  
    g.setColor(i, size-1-i, Color.RED);
```



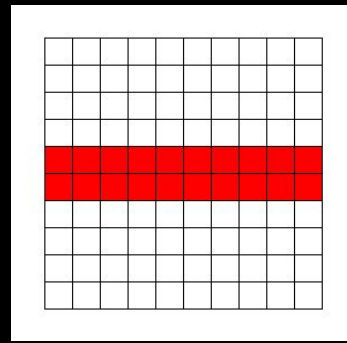
A



B



C



D

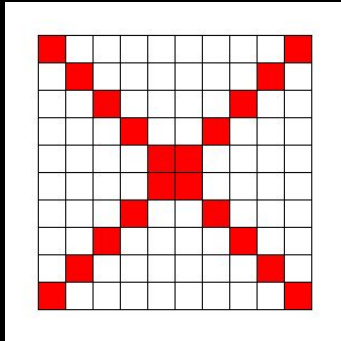
E: none of the above

What will this draw?

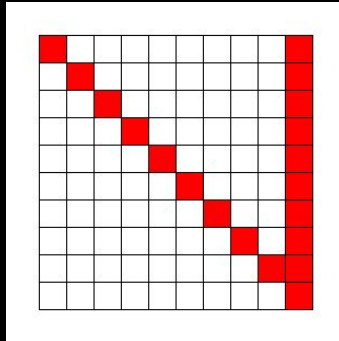
```
int size = 10;  
SimpleGrid g=new SimpleGrid(size, size);  
for (int i=0; i < size; i++)  
    g.setColor(i, i, Color.RED);  
g.setColor(i, size-1-i, Color.RED);
```

This is what the compiler reads;
in Java, indentation is only for
humans.

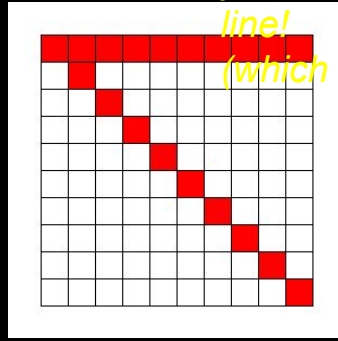
Compiler: *If you omit curly braces,
you must only mean to group one
line!*
(which ends with a semicolon)



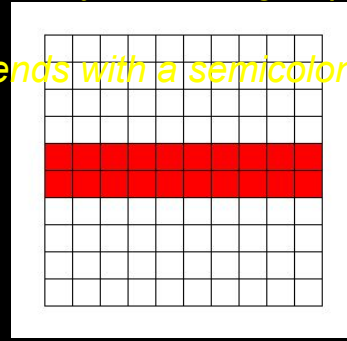
A



B



C



D

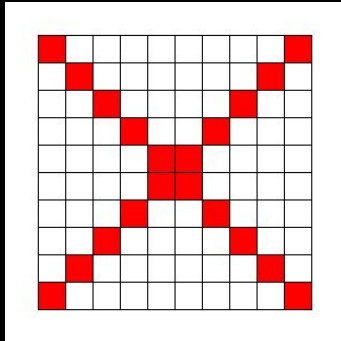
E: none of the above

What will this draw?

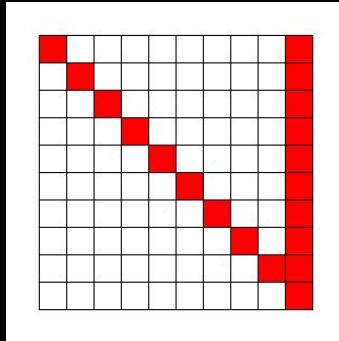
```
int size = 10;  
SimpleGrid g=new SimpleGrid(size, size);  
for (int i=0; i < size; i++) {  
    g.setColor(i, i, Color.RED);  
    g.setColor(i, size-1-i, Color.RED);  
}
```

missing curly braces to group
everything inside for-loop;
as is, only the first setColor() will loop!

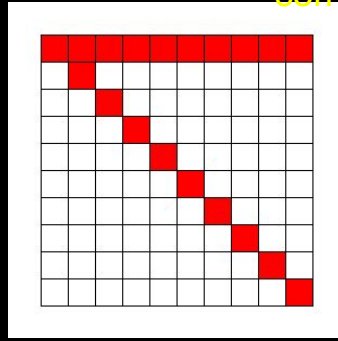
second setColor() won't even
compile...



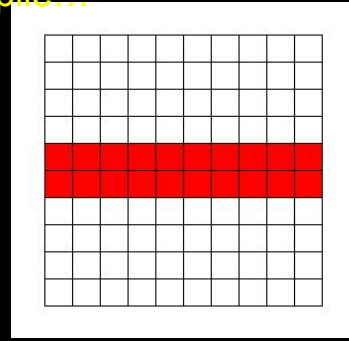
A



B



C



D

E: none of the above

Without curly braces, for loops
just repeat the first statement!

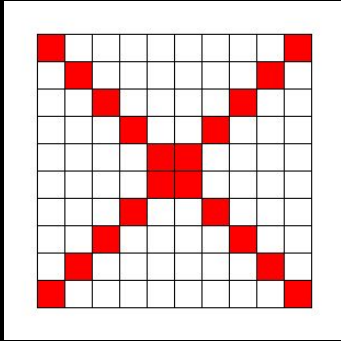
```
for(int i=0; i<3; i++)  
    System.out.println("message1");  
    System.out.println("message2");
```

This prints:

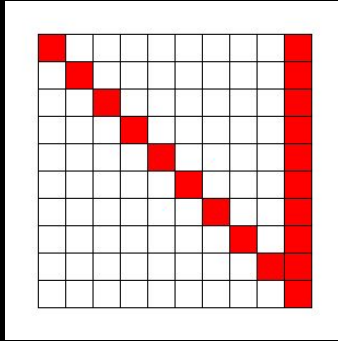
```
message1  
message1  
message1  
message2
```


OK, how about now?

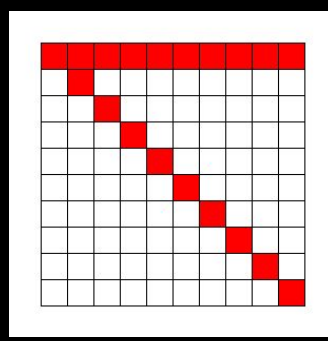
```
int size = 10;  
SimpleGrid g=new SimpleGrid(size, size);  
for (int i=0; i < size; i++) {  
    g.setColor(i, i, Color.RED);  
    g.setColor(i, size-1-i, Color.RED);  
}
```



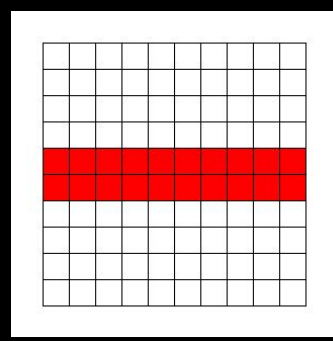
A



B



C



D

E: none of the above

OK, how about now?

```
int size = 10;  
SimpleGrid g=new SimpleGrid(size, size);  
for (int i=0; i < size; i++);
```

these
do
nothing
here

X

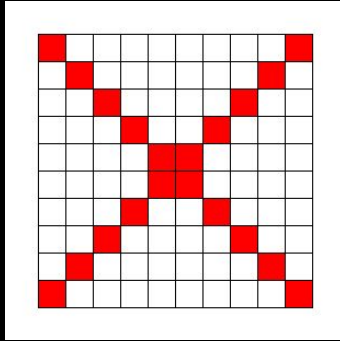
```
g.setColor(i, i, Color.RED);  
g.setColor(i, size-1-i, Color.RED);
```

X

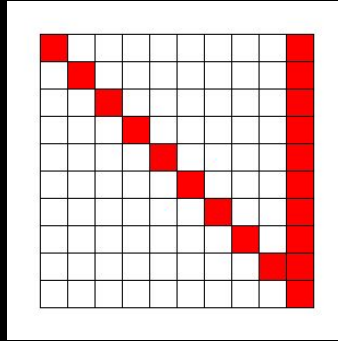
This is what the compiler reads:
do nothing size-times!

Compiler: *If you omit curly braces,
you must only mean to group one
line!*

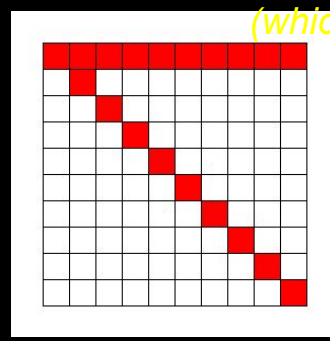
(which ends with a semicolon)



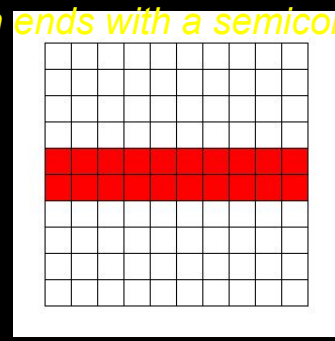
A



B



C



D

E: none of the above